You said:

lets remember all conversation and approved build plan to start building

Worked for 36s

ChatGPT said:

[Certain] The project history is available in the current conversation and its project files, but **we should not start coding from a mixture of proposed features and approved requirements.** We need one authoritative build specification that distinguishes confirmed decisions, proposed architecture, and unresolved client approvals.

I've consolidated the working baseline for **Kalyan Hospital — Offline Patient & Physiotherapy Management Software** below. This is the reference we should use to start development without losing earlier decisions.

Kalyan Hospital Software — Master Build Plan

Kalyan Hospital

Patient & Clinical Management System

श्री वशिष्ठ कल्याण ट्रस्ट · Ara, Bihar

Client budget ceiling	Target delivery
₹22,000	4–6 weeks
Operation	**Deployment proposal**
Fully offline	**Local network (LAN)**

Budget and timeline are project constraints. The final scope, technical stack and deployment setup still require confirmation against the hospital's actual requirements.

1. What we are building

[Certain] The objective is a hospital-specific application that stores patient records locally, supports clinical workflows and billing, and continues operating without an internet connection.

[Likely] The most recent architecture proposal is a **centralized local application**: one server on the hospital's premises, with authorized reception and doctor computers accessing the same database over the local network. This is different from installing independent copies on every computer.

Patient and reception management

Patient registration, unique URN, patient search, profile editing, photographs, appointments and doctor assignment.

Clinical records

Doctor-specific encounters, consultation notes, prescriptions, treatment plans and physiotherapy session history.

Billing and treatment packages

Consultation charges, payments, outstanding balances, receipts and session-package tracking.

Administration and data protection

Individual logins, permissions, audit events, local backup, restore and data-integrity checks.

2. Technical decisions we will preserve

[Likely] The centralized LAN architecture is the leading option because shared records across multiple computers are part of the intended use case. We must confirm the actual hardware and network before committing to it.

Component	Proposed choice
Frontend	React + TypeScript
Backend	Python + FastAPI
Database	PostgreSQL
Deployment	Local hospital server + LAN
Patient photos and documents	Local storage
Security	Individual accounts, backend-enforced permissions

Data protection	Audit events, backup and tested restore
Internet	Not required for normal operation

The earlier alternative, PySide6 + SQLite, remains relevant only if we confirm a single-computer deployment. We should not mix the two architectures.

3. Open-source projects to learn from

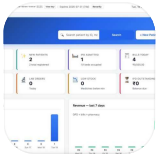
These are the repository references already identified in the project history.



1. Open Hospital

Study desktop deployment, patient records and LAN architecture.

GitHub repository ↗



2. ICTHospital

Study billing, permissions and module boundaries.

GitHub repository ↗



3. OpenEMR

Study clinical encounters, medical history and billing workflows.

GitHub repository ↗



4. GNU Health

Study patient, encounter and admission data models.

Documentation ↗



5. VITALIt OS

Study the FastAPI-based architecture and modern interface approach.

GitHub repository ↗

[Certain] These are research references, not proof that their code has been audited or is ready to reuse. We must verify licensing, dependencies and actual offline behavior before adopting code.

4. Build sequence

1. Inspect the supplied specifications, images and project notes.
2. Confirm the exact client-approved MVP scope and workstation requirements.
3. Review the repositories and choose one architecture.
4. Design the database, migrations and authorization model.
5. Build login, hospital settings, doctor profiles, patients and appointments.
6. Build clinical encounters, treatment sessions, and agreed billing features.
7. Implement printing, reports, audit events and backup/restore.
8. Test offline operation, data integrity, permissions and recovery before handover.

5. Project memory has been updated

Master project record updated

[Certain] I added the build kickoff baseline to the existing project record in your connected Engram knowledge base:

`Projects/Kalyan Hospital Software/Kalyan Hospital Software — Single Source of Truth.md`

It now includes the project constraints, architecture proposal, repository links, build sequence, exclusions, commercial guardrails and unresolved client decisions.

The separate Noteithub journal save was blocked by its credit limit, so I have not marked that save as successful.

Our next action should be to inspect the uploaded `Hospital Software Specification.txt` and related project files, reconcile their requirements, and establish the definitive build specification before writing application code. The existing project record is our baseline; client approval of every proposed feature is still outstanding.